MCA LAB

UART and ADC Programs – Code Snippets and Expected Outputs

List of Programs:

1. UART: Display text by transmitting data to the console at any baud rate. The LED must glow as many times as the text is displayed.

    a. Glow the LED once each time the text is displayed, and so on.

    b. Modify (a): if the character 'A' exists in the text, glow the LED twice.

    c. Modify (b): change the cases vice versa.

2. UART: Using a look-up table of the multiplication table of 2, if a number is present in the table, display on the console how many times 2 has been multiplied to get it. (Existing code is deleted and own UART set-up code is written.)

3. ADC: Read the internal temperature of the controller and display it on the console.

4. ADC: Vary the temperature using a potentiometer (POT) and display it on the console.

Board: STM32F103C8T6 (Blue Pill)

Program 1: UART Text Transmission with LED Indication

Problem Statement

Display text on the console by transmitting the data through UART with any baud rate. If the text is displayed once, glow the LED once, and so on.

- Modify the above program: if the character 'A' exists in the text, glow the LED twice.
- Modify Program 1: change the cases vice versa.

1a. Text Display with LED Glow Count

Code Snippets:



```c
#include <libopencm3/stm32/rcc.h>
#include <libopencm3/stm32/gpio.h>
#include <libopencm3/stm32/usart.h>

static void gpio_setup(void)
{
    rcc_periph_clock_enable(RCC_GPIOA);

    /* PA0 as output */
    gpio_set_mode(GPIOA,
                  GPIO_MODE_OUTPUT_2_MHZ,
                  GPIO_CNF_OUTPUT_PUSHPULL,
                  GPIO0);
}

static void usart_setup(void)
{
    rcc_periph_clock_enable(RCC_USART1);

    /* PA9 = USART1 TX */
    gpio_set_mode(GPIOA,
                  GPIO_MODE_OUTPUT_50_MHZ,
                  GPIO_CNF_OUTPUT_ALTFN_PUSHPULL,
                  GPIO_USART1_TX);

    usart_set_baudrate(USART1, 9600);
    usart_set_databits(USART1, 8);
    usart_set_stopbits(USART1, USART_STOPBITS_1);
    usart_set_mode(USART1, USART_MODE_TX);
    usart_set_parity(USART1, USART_PARITY_NONE);
    usart_set_flow_control(USART1, USART_FLOWCONTROL_NONE);

    usart_enable(USART1);
}

static void usart_send_string(const char *str)
{
    while (*str)
    {
        usart_send_blocking(USART1, *str);
        str++;
    }
}

static void delay(void)
{
    for (volatile int i = 0; i < 1500000; i++)
        __asm__("nop");
}

static void led_blink(int count)
{
    for (int i = 0; i < count; i++)
    {
        gpio_set(GPIOA, GPIO0);
        delay();

        gpio_clear(GPIOA, GPIO0);
        delay();
    }
}

int main(void)
{
    rcc_clock_setup_in_hse_8mhz_out_24mhz();

    gpio_setup();
    usart_setup();

    const char text[] = "MANIPAL";

    while (1)
    {
        /* Transmit text */
        usart_send_string(text);
        usart_send_blocking(USART1, '\r');
        usart_send_blocking(USART1, '\n');

        /* Blink LED once */
        led_blink(1);

        delay();
    }

    return 0;
}
```

Fig: UART Text Display C code snippet

Expected Output:

The text is transmitted through UART at the selected baud rate (e.g. 9600 bps, 8 data bits, no parity, 1 stop bit) and is displayed on the serial terminal.

Each time the text is displayed, the LED glows once, so the LED glows as many times as the text is displayed.

Console (serial terminal) output:

```
Hello from STM32
Hello from STM32
Hello from STM32
```

Text displayed	Lines on console	LED glows
Once	1	Once
Twice	2	Twice
Thrice	3	Thrice

1b. LED Glows Twice if Character 'A' Exists

Code Snippets:

```c
#include <libopencm3/stm32/rcc.h>
#include <libopencm3/stm32/gpio.h>
#include <libopencm3/stm32/usart.h>

static void gpio_setup(void)
{
    rcc_periph_clock_enable(RCC_GPIOA);

    /* PA0 as output */
    gpio_set_mode(GPIOA,
                  GPIO_MODE_OUTPUT_2_MHZ,
                  GPIO_CNF_OUTPUT_PUSHPULL,
                  GPIO0);
}

static void usart_setup(void)
{
    rcc_periph_clock_enable(RCC_USART1);

    /* PA9 = USART1 TX */
    gpio_set_mode(GPIOA,
                  GPIO_MODE_OUTPUT_50_MHZ,
                  GPIO_CNF_OUTPUT_ALTFN_PUSHPULL,
                  GPIO_USART1_TX);

    usart_set_baudrate(USART1, 9600);
    usart_set_databits(USART1, 8);
    usart_set_stopbits(USART1, USART_STOPBITS_1);
    usart_set_mode(USART1, USART_MODE_TX);
    usart_set_parity(USART1, USART_PARITY_NONE);
    usart_set_flow_control(USART1, USART_FLOWCONTROL_NONE);

    usart_enable(USART1);
}

static void usart_send_string(const char *str)
{
    while (*str)
    {
        usart_send_blocking(USART1, *str);
        str++;
    }
}

static void delay(void)
{
    for (volatile int i = 0; i < 1500000; i++)
        __asm__("nop");
}

static void led_blink(int count)
{
    for (int i = 0; i < count; i++)
    {
        gpio_set(GPIOA, GPIO0);
        delay();

        gpio_clear(GPIOA, GPIO0);
        delay();
    }
}

int main(void)
{
    rcc_clock_setup_in_hse_8mhz_out_24mhz();

    gpio_setup();
    usart_setup();

    const char text[] = "MANIPAL";

    while (1)
    {
        /* Transmit text */
        usart_send_string(text);
        usart_send_blocking(USART1, '\r');
        usart_send_blocking(USART1, '\n');

        /* Blink LED once */
        led_blink(1);

        delay();
    }

    return 0;
}
```

Fig: Character A Check C code snippet

Expected Output:

The text is displayed on the console. The program checks every character of the text.

If the character 'A' is present in the text, the LED glows twice; otherwise the LED glows once.

Console (serial terminal) output:

```
ARM CONTROLLER   -> 'A' found     -> LED glows twice
HELLO WORLD      -> 'A' not found -> LED glows once
```

Text	'A' present?	LED glows
ARM CONTROLLER	Yes	Twice
HELLO WORLD	No	Once

1c. Cases Reversed (Vice Versa)

Code Snippets:

```c
#include <libopencm3/stm32/rcc.h>
#include <libopencm3/stm32/gpio.h>
#include <libopencm3/stm32/usart.h>

static void gpio_setup(void)
{
    rcc_periph_clock_enable(RCC_GPIOA);

    /* PA0 as output */
    gpio_set_mode(GPIOA,
                  GPIO_MODE_OUTPUT_2_MHZ,
                  GPIO_CNF_OUTPUT_PUSHPULL,
                  GPIO0);
}

static void usart_setup(void)
{
    rcc_periph_clock_enable(RCC_USART1);

    /* PA9 = USART1 TX */
    gpio_set_mode(GPIOA,
                  GPIO_MODE_OUTPUT_50_MHZ,
                  GPIO_CNF_OUTPUT_ALTFN_PUSHPULL,
                  GPIO_USART1_TX);

    usart_set_baudrate(USART1, 9600);
    usart_set_databits(USART1, 8);
    usart_set_stopbits(USART1, USART_STOPBITS_1);
    usart_set_mode(USART1, USART_MODE_TX);
    usart_set_parity(USART1, USART_PARITY_NONE);
    usart_set_flow_control(USART1, USART_FLOWCONTROL_NONE);

    usart_enable(USART1);
}

static void usart_send_string(const char *str)
{
    while (*str)
    {
        usart_send_blocking(USART1, *str);
        str++;
    }
}

static void delay(void)
{
    for (volatile int i = 0; i < 1500000; i++)
        __asm__("nop");
}

static void led_blink(int count)
{
    for (int i = 0; i < count; i++)
    {
        gpio_set(GPIOA, GPIO0);
        delay();

        gpio_clear(GPIOA, GPIO0);
        delay();
    }
}

int main(void)
{
    rcc_clock_setup_in_hse_8mhz_out_24mhz();

    gpio_setup();
    usart_setup();

    const char text[] = "MANIPAL";

    while (1)
    {
        /* Transmit text */
        usart_send_string(text);
        usart_send_blocking(USART1, '\r');
        usart_send_blocking(USART1, '\n');

        /* Blink LED once */
        led_blink(1);

        delay();
    }

    return 0;
}
```

Fig: Reversed Cases C code snippet

Expected Output:

The cases of Program 1b are reversed: if the character 'A' is present in the text, the LED glows once; if 'A' is not present, the LED glows twice.

Console (serial terminal) output:

```
ARM CONTROLLER   -> 'A' found     -> LED glows once
HELLO WORLD      -> 'A' not found -> LED glows twice
```

Text	'A' present?	LED glows
ARM CONTROLLER	Yes	Once
HELLO WORLD	No	Twice

Program 2: Multiplication Table of 2 Using a Look-up Table

Problem Statement

Consider a look-up table of the multiplication table of 2. If any number is present in the table, show how many times 2 has been multiplied to get it and display it on the console.

Note: The existing code is deleted and own set-up code for the UART is written (clock enable, GPIO configuration of TX/RX pins, baud rate and UART enable).

Code Snippets:

```c
#include <libopencm3/stm32/rcc.h>
#include <libopencm3/stm32/gpio.h>

static void gpio_setup(void)
{
    /* Enable GPIOA clock */
    rcc_periph_clock_enable(RCC_GPIOA);

    /* PA0 and PA1 as output push-pull */
    gpio_set_mode(GPIOA,
                  GPIO_MODE_OUTPUT_2_MHZ,
                  GPIO_CNF_OUTPUT_PUSHPULL,
                  GPIO0 | GPIO1);

    /* PA3 as input with pull-up for button */
    gpio_set_mode(GPIOA,
                  GPIO_MODE_INPUT,
                  GPIO_CNF_INPUT_PULL_UPDOWN,
                  GPIO3);

    /* Enable internal pull-up */
    gpio_set(GPIOA, GPIO3);
}

int main(void)
{
    int count = 0;

    gpio_setup();

    while (1)
    {
        /* Detect button press */
        if (gpio_get(GPIOA, GPIO3) == 0)
        {
            /* Wait until button is released */
            while (gpio_get(GPIOA, GPIO3) == 0)
            {
                // Wait
            }

            /* Button has been released */
            count++;

            if (count == 1)
            {
                /* 1st press: LED1 ON, LED2 OFF */
                gpio_set(GPIOA, GPIO0);
                gpio_clear(GPIOA, GPIO1);
            }
            else if (count == 2)
            {
                /* 2nd press: Both LEDs ON */
                gpio_set(GPIOA, GPIO0);
                gpio_set(GPIOA, GPIO1);
            }
            else if (count == 3)
            {
                /* 3rd press: LED1 OFF, LED2 ON */
                gpio_clear(GPIOA, GPIO0);
                gpio_set(GPIOA, GPIO1);

                /* Reset for next cycle */
                count = 0;
            }
        }
    }

    return 0;
}
```

Fig: Look-up Table UART C code snippet

Expected Output:

The look-up table holds 2, 4, 6, … 20 (2 × 1 to 2 × 10). The number to be checked is compared with every entry of the table.

If the number is found, the console shows the multiplier (how many times 2 has been multiplied). If it is not found, a 'not in table' message is displayed.

Console (serial terminal) output:

```
Number : 8
8 is in the table of 2 -> 2 x 4 = 8 (multiplied 4 times)

Number : 14
14 is in the table of 2 -> 2 x 7 = 14 (multiplied 7 times)

Number : 15
15 is not in the table of 2
```

Number	Found in table?	Console message
8	Yes	2 x 4 = 8 (multiplied 4 times)
14	Yes	2 x 7 = 14 (multiplied 7 times)
20	Yes	2 x 10 = 20 (multiplied 10 times)
15	No	15 is not in the table of 2

Program 3: ADC – Internal Temperature of the Controller

Problem Statement

Write a program for the ADC to access the internal temperature of the controller and display it on the console.

Code Snippets:

```c
#include <libopencm3/stm32/rcc.h>
#include <libopencm3/stm32/gpio.h>

static void gpio_setup(void)
{
    /* Enable GPIOA clock */
    rcc_periph_clock_enable(RCC_GPIOA);

    /* PA0 and PA1 as output push-pull */
    gpio_set_mode(GPIOA,
                  GPIO_MODE_OUTPUT_2_MHZ,
                  GPIO_CNF_OUTPUT_PUSHPULL,
                  GPIO0 | GPIO1);

    /* PA3 as input with pull-up for button */
    gpio_set_mode(GPIOA,
                  GPIO_MODE_INPUT,
                  GPIO_CNF_INPUT_PULL_UPDOWN,
                  GPIO3);

    /* Enable internal pull-up */
    gpio_set(GPIOA, GPIO3);
}

int main(void)
{
    int count = 0;

    gpio_setup();

    while (1)
    {
        /* Detect button press */
        if (gpio_get(GPIOA, GPIO3) == 0)
        {
            /* Wait until button is released */
            while (gpio_get(GPIOA, GPIO3) == 0)
            {
                // Wait
            }

            /* Button has been released */
            count++;

            if (count == 1)
            {
                /* 1st press: LED1 ON, LED2 OFF */
                gpio_set(GPIOA, GPIO0);
                gpio_clear(GPIOA, GPIO1);
            }
            else if (count == 2)
            {
                /* 2nd press: Both LEDs ON */
                gpio_set(GPIOA, GPIO0);
                gpio_set(GPIOA, GPIO1);
            }
            else if (count == 3)
            {
                /* 3rd press: LED1 OFF, LED2 ON */
                gpio_clear(GPIOA, GPIO0);
                gpio_set(GPIOA, GPIO1);

                /* Reset for next cycle */
                count = 0;
            }
        }
    }

    return 0;
}
```

Fig: Internal Temperature ADC C code snippet

Expected Output:

The internal temperature sensor (ADC1 channel 16) is read and the ADC value (0–4095) is converted into temperature using T (°C) = ((V25 − Vsense) / Avg_Slope) + 25, where V25 ≈ 1.43 V and Avg_Slope ≈ 4.3 mV/°C (typical values from the datasheet).

The ADC value and the temperature are displayed on the console continuously. The value is close to room temperature and rises slowly when the chip is touched or warms up.

Console (serial terminal) output:

```
ADC Value : 1739   Temperature : 31.6 C
ADC Value : 1737   Temperature : 32.0 C
ADC Value : 1734   Temperature : 32.6 C
```

Note: The console values shown are typical examples; the actual reading varies from chip to chip (the internal sensor has an offset of a few °C).

Program 4: ADC – Vary the Temperature Using a POT

Problem Statement

Write a program for the ADC where a potentiometer (POT) is used to vary the temperature, and display it on the console.

Code Snippets:

```c
#include <stdint.h>
#include <stdio.h>

#include <libopencm3/stm32/rcc.h>
#include <libopencm3/stm32/gpio.h>
#include <libopencm3/stm32/adc.h>
#include <libopencm3/stm32/usart.h>

int main(void)
{
    uint16_t adc_value;
    float voltage;
    float temperature;
    char buffer[100];

    /* Configure system clock */
    rcc_clock_setup_in_hse_8mhz_out_24mhz();

    /* Enable GPIOA, ADC1 and USART1 clocks */
    rcc_periph_clock_enable(RCC_GPIOA);
    rcc_periph_clock_enable(RCC_ADC1);
    rcc_periph_clock_enable(RCC_USART1);

    /* PA0 as analog input */
    gpio_set_mode(GPIOA,
                  GPIO_MODE_INPUT,
                  GPIO_CNF_INPUT_ANALOG,
                  GPIO0);

    /* PA9 as USART1 TX */
    gpio_set_mode(GPIOA,
                  GPIO_MODE_OUTPUT_50_MHZ,
                  GPIO_CNF_OUTPUT_ALTFN_PUSHPULL,
                  GPIO9);

    /* Configure USART1 */
    usart_set_baudrate(USART1, 9600);
    usart_set_databits(USART1, 8);
    usart_set_stopbits(USART1, USART_STOPBITS_1);
    usart_set_mode(USART1, USART_MODE_TX);
    usart_set_parity(USART1, USART_PARITY_NONE);
    usart_set_flow_control(USART1, USART_FLOWCONTROL_NONE);

    usart_enable(USART1);

    /* ADC clock = PCLK2 / 6 */
    rcc_set_adcpre(RCC_CFGR_ADCPRE_PCLK2_DIV6);

    /* Reset ADC1 */
    rcc_periph_reset_pulse(RST_ADC1);

    /* Independent ADC mode */
    adc_set_dual_mode(ADC_CR1_DUALMOD_IND);

    /* Disable scan mode */
    adc_disable_scan_mode(ADC1);

    /* Single conversion mode */
    adc_set_single_conversion_mode(ADC1);

    /* Select ADC channel 0 (PA0) */
    uint8_t channels[] = {ADC_CHANNEL0};
    adc_set_regular_sequence(ADC1, 1, channels);

    /* Set sampling time */
    adc_set_sample_time(ADC1, ADC_CHANNEL0,
                        ADC_SMPR_SMP_239DOT5CYC);

    /* Enable software triggering */
    adc_enable_external_trigger_regular(
        ADC1, ADC_CR2_EXTSEL_SWSTART);

    /* Power on ADC */
    adc_power_on(ADC1);

    /* Wait for stabilization */
    for (volatile int i = 0; i < 10000; i++)
        __asm__("nop");

    /* Calibrate ADC */
    adc_reset_calibration(ADC1);
    adc_calibrate(ADC1);

    while (1)
    {
        /* Start ADC conversion */
        adc_start_conversion_regular(ADC1);

        /* Wait for conversion to complete */
        while (!adc_eoc(ADC1));

        /* Read ADC result */
        adc_value = adc_read_regular(ADC1);

        /* Convert ADC value to voltage */
        voltage = (adc_value * 3.3f) / 4095.0f;

        /* Scale voltage to temperature range 0–100 °C */
        temperature = (adc_value * 100.0f) / 4095.0f;

        /* Display ADC value, voltage and temperature */
        snprintf(buffer, sizeof(buffer),
                 "ADC Value : %u   Voltage : %.2f V   Temperature : %.0f C\r\n",
                 adc_value, voltage, temperature);

        for (int i = 0; buffer[i] != '\0'; i++)
        {
            usart_send_blocking(USART1, buffer[i]);
        }

        /* Small delay */
        for (volatile int i = 0; i < 500000; i++)
            __asm__("nop");
    }

    return 0;
}
```

Fig: POT Temperature ADC C code snippet

Expected Output:

The POT wiper is connected to an ADC input pin (e.g. PA0 – ADC1 channel 0) with its ends connected to 3.3 V and GND. The ADC value (0–4095) is scaled to a temperature range (e.g. 0–100 °C) and displayed on the console.

Rotating the POT changes the voltage at the pin, so the displayed temperature increases or decreases accordingly.

Console (serial terminal) output:

```
ADC Value : 0      Voltage : 0.00 V   Temperature : 0 C
ADC Value : 2048   Voltage : 1.65 V   Temperature : 50 C
ADC Value : 4095   Voltage : 3.30 V   Temperature : 100 C
```

POT position	ADC value	Voltage	Temperature
Minimum	0	0.00 V	0 °C
Middle	≈ 2048	≈ 1.65 V	≈ 50 °C
Maximum	4095	3.30 V	100 °C